UNIVERSIDAD NACIONAL DE LOJA
Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables
Carrera de Computación · Ciclo 6
Teoría de Autómatas y Computabilidad Avanzada

Evaluación Sumativa 1 · Unidad 1

Monitor de intentos de pago en la compra de boletos de cine con autómatas finitos

Informe técnico

Equipo: Monitor Cine Autómatas

Integrante	Rol
Jossibel Pérez	Documentadora (informe y presentación)
David Toledo	Responsable del modelo formal (AFND, AFD y JFLAP)
Isauro Rivera	Responsable de datos y pruebas
Jean Encalada	Desarrollador del monitor

Docente: Ing. Genoveva Suing Albito
Repositorio: https://github.com/Jean1204-code/monitor-cine-automatas
Loja, 28 de octubre de 2026

1. Introducción

Cuando alguien compra una entrada de cine desde una aplicación, cada intento de pago queda registrado con su resultado: la orden salió bien, el banco la rechazó, no hubo respuesta del banco, faltaron fondos, etc. Para este proyecto usamos dos archivos publicados de un sistema de venta de boletos: ticket_history.csv, con 154 827 operaciones entre 2019 y 2022, y status_detail.csv, que explica los ocho resultados posibles de cada operación [2], [3]. Cada registro tiene un cliente, una fecha y un resultado. No trae los pasos internos de la compra (elegir función, butaca, etc.), así que el monitor trabaja sobre los resultados de pago.

La pregunta guía es qué secuencias de resultados conviene vigilar y cómo puede detectarlas un autómata finito. Elegimos tres: una racha de tres pagos fallidos, una compra exitosa justo después de dos fallos y dos faltas de respuesta del banco seguidas de un fallo. Las tratamos como alertas para revisión, no como fraude confirmado.

El objetivo es construir y evaluar un monitor basado en un AFND por patrón y en el AFD mínimo equivalente. Para eso preparamos los datos de forma reproducible, formalizamos los patrones, hicimos las construcciones (unión con ε, eliminación de ε, subconjuntos y minimización), medimos el monitor y documentamos sus límites [1], [4].

Al principio probamos con otro archivo (cinemaTicket_Ref.csv), pero solo traía ventas totales y no resultados individuales, así que no servía para formar cadenas.

2. Modelo formal

2.1 Alfabeto

El alfabeto sale directamente del archivo status_detail.csv: cada código de resultado es un símbolo, así que no inventamos variables. Σ = {S, T, F, B, L, P, V, R}.

status_id	Símbolo	Resultado según status_detail.csv	Relevancia
1	S	Orden exitosa (Order successful)	Relevante
-1	T	Pago vencido (Payment overdue)	Relevante
-2	F	Fondos insuficientes (Insufficient funds…)	Relevante
-3	B	Sin respuesta del banco (No response from your bank)	Relevante
-4	L	Contraseña bloqueada (Password locked…)	Relevante
-5	P	Pago fallido desde el banco (Payment failed from bank)	Relevante
-6	V	Se requiere verificar la cuenta (Need verify your account…)	Irrelevante (bucle)
-7	R	Transacción temporalmente limitada (Transaction temporarily limited)	Relevante

Tabla 1. Diccionario de eventos.

V es el evento irrelevante: es un pedido de la aplicación al cliente para que verifique su cuenta, no un resultado del banco, y por eso no debería romper ni completar una racha de resultados bancarios. Formalmente usamos una proyección π que borra las V y deja intactos los demás símbolos; en los autómatas, V es un bucle en los estados intermedios.

Cada caso es un cliente en un año calendario (cliente-año), con sus operaciones ordenadas por fecha. La guía permite agrupar por usuario; elegimos el año para que los historiales no mezclen operaciones demasiado lejanas en el tiempo. Por ejemplo, un cliente con un fallo del banco, una verificación de cuenta, otro fallo y una compra exitosa forma la cadena w = PVPS, con π(w) = PPS.

2.2 Patrones

$L_1 = \{w \in \Sigma^* \mid \pi(w) \text{ contiene PPP}\}$, $L_2 = \{w \in \Sigma^* \mid \pi(w) \text{ contiene PPS}\}$ y $L_3 = \{w \in \Sigma^* \mid \pi(w) \text{ contiene BBP}\}$. El monitor acepta $L = L_1 \cup L_2 \cup L_3$; aceptar significa levantar una alerta, no autorizar una compra.

Patrón	Qué indica	Pertenecen	No pertenecen
P1: PPP	Racha de tres pagos fallidos desde el banco	PPP, PVPP	PP, PFP
P2: PPS	Compra exitosa justo después de dos fallos	PPS, PPVS	PS, PTS
P3: BBP	Dos faltas de respuesta del banco seguidas de un fallo	BBP, BVBVP	BB, BSP

Tabla 2. Patrones del monitor y cadenas de prueba.

Por qué estos patrones. P1 y P2 siguen lo que describen fuentes técnicas sobre la prueba de tarjetas robadas: OWASP la define como «múltiples intentos de autorización de pago usados para verificar la validez de datos de tarjetas robadas» [5], y Stripe explica que se reconoce por «un aumento significativo de autorizaciones y pagos fallidos» y que, una vez que el atacante identifica las tarjetas válidas, las usa para comprar [6]. Por eso una racha de fallos (P1) y una compra que sale bien justo después de varios fallos (P2) merecen revisión. P3 es un indicador operativo nuestro: si el banco no responde dos veces y después falla, puede haber un problema con la pasarela. Los umbrales (tres y dos) los fijamos nosotros; las fuentes no dan un número.

Una vez que se cumple un patrón, cualquier continuación sigue aceptada. S, T, F, L y R cortan una coincidencia parcial cuando no la continúan; V la mantiene.

3. Conjunto de datos

Descargamos ticket_history.csv y status_detail.csv desde la carpeta enlazada en el repositorio Movie_Ticket_Python [2], [3]. El repositorio describe una aplicación de venta de entradas, pero no indica quién es la empresa ni bajo qué licencia se publican los datos; esa limitación se discute en la sección 9. El script datos/preparar_datos.py hace todo el procesamiento con semilla fija (2026), así que los archivos se pueden regenerar idénticos:

- **Limpieza.** Se quitan 102 filas duplicadas exactas; quedan 154 725 operaciones de 119 477 clientes.
- **Casos.** Se ordena por cliente, fecha y ticket_id, se agrupa por cliente y año y se conservan los grupos de 3 a 20 operaciones, sin recortarlos: 3 937 casos.
- **Etiquetas.** Se calculan con una búsqueda de subcadenas sobre π(w), independiente de los autómatas. En los 3 937 casos hay 153 con alerta (3,89 %).
- **Muestreo.** Como la proporción natural de alertas es baja, se arma una muestra enriquecida al 18 % (dentro del 15-20 % que pide la guía), con un solo caso por cliente y sin clientes repetidos entre validación y prueba. P3 es muy escaso, así que se aseguraron dos casos en validación y uno en prueba. No se inventó ninguna operación.

Archivo	Casos	Eventos	Alertas	Contenido
validacion.csv	300	1 047	54 (18 %)	Con etiqueta y tipo_patron
prueba.csv	50	187	9 (18 %)	Sin etiquetas (la clave va aparte)

Tabla 3. Archivos del proyecto.

Las columnas son id_evento, fecha_hora, id_sesion (el caso cliente-año), actor (código local que reemplaza al customer_id), evento y simbolo; validación agrega etiqueta y tipo_patron. En los casos seleccionados P1 aparece 16 veces, P2 47 y P3 3 (pueden coincidir en un mismo caso). Hay también casos casi anómalos, como PP o BB sin el tercer símbolo, y casos con V en medio de una racha.

4. Diseño de los AFND

Los tres AFND tienen la misma forma. Para k = 1, 2, 3 y el patrón $a_1a_2a_3$ (PPP, PPS o BBP): **M_k = ({p_k0, p_k1, p_k2, p_k3}, Σ, δ_k, p_k0, {p_k3})**. p_k0 significa «todavía no empieza una coincidencia»; p_k1 y p_k2 recuerdan que ya se leyó el primer símbolo o los dos primeros; p_k3, que el patrón se completó. V es un bucle en p_k1 y p_k2; cualquier otro símbolo que no continúe el patrón mata ese camino.

Son no deterministas porque en p_k0, al leer a_1, el autómata no sabe si ese símbolo inicia el patrón o es uno suelto, así que sigue en p_k0 y a la vez pasa a p_k1. Cada AFND tiene 4 estados y 21 transiciones. Las figuras son las capturas de JFLAP; las tablas, sus transiciones.

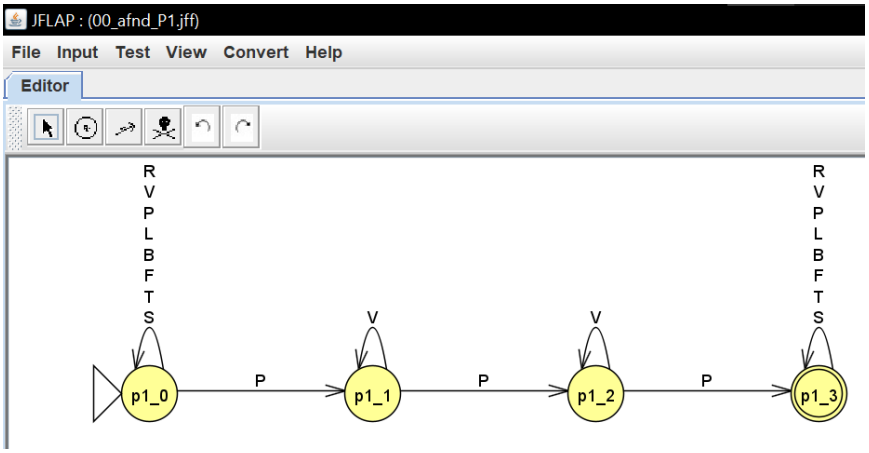


Figura 1. AFND de P1 (PPP) en JFLAP.

δ	S	T	F	B	L	P	V	R
→p1_0	{p1_0}	{p1_0}	{p1_0}	{p1_0}	{p1_0}	{p1_0, p1_1}	{p1_0}	{p1_0}
p1_1	ø	ø	ø	ø	ø	{p1_2}	{p1_1}	ø
p1_2	ø	ø	ø	ø	ø	{p1_3}	{p1_2}	ø
*p1_3	{p1_3}	{p1_3}	{p1_3}	{p1_3}	{p1_3}	{p1_3}	{p1_3}	{p1_3}

Tabla 4. Transiciones de M_1.

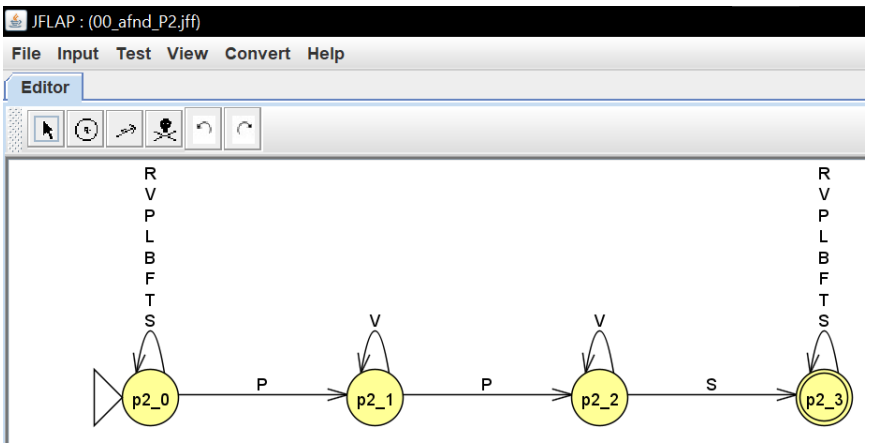


Figura 2. AFND de P2 (PPS) en JFLAP.

δ	S	T	F	B	L	P	V	R
→p2_0	{p2_0}	{p2_0}	{p2_0}	{p2_0}	{p2_0}	{p2_0, p2_1}	{p2_0}	{p2_0}
p2_1	ø	ø	ø	ø	ø	{p2_2}	{p2_1}	ø
p2_2	{p2_3}	ø	ø	ø	ø	ø	{p2_2}	ø
*p2_3	{p2_3}	{p2_3}	{p2_3}	{p2_3}	{p2_3}	{p2_3}	{p2_3}	{p2_3}

Tabla 5. Transiciones de M_2.

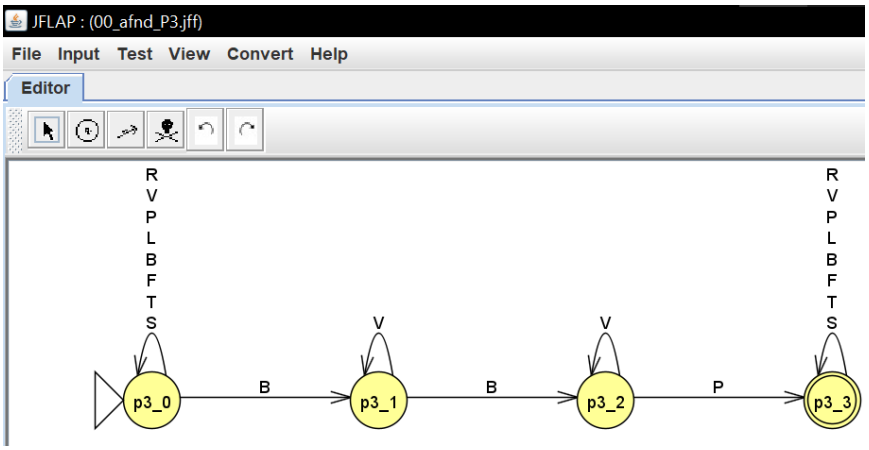


Figura 3. AFND de P3 (BBP) en JFLAP.

δ	S	T	F	B	L	P	V	R
→p3_0	{p3_0}	{p3_0}	{p3_0}	{p3_0, p3_1}	{p3_0}	{p3_0}	{p3_0}	{p3_0}
p3_1	∅	∅	∅	{p3_2}	∅	∅	{p3_1}	∅
p3_2	∅	∅	∅	∅	∅	{p3_3}	{p3_2}	∅
*p3_3	{p3_3}	{p3_3}	{p3_3}	{p3_3}	{p3_3}	{p3_3}	{p3_3}	{p3_3}

Tabla 6. Transiciones de M_3.

5. Unión con transiciones ε y su eliminación

Para tener un solo autómata agregamos un estado inicial i con tres transiciones ε hacia p1_0, p2_0 y p3_0. M_ε tiene 13 estados, 66 transiciones (tres de ellas ε) y F = {p1_3, p2_3, p3_3}. La ε no representa un evento real; es la construcción matemática para unir lenguajes [4].

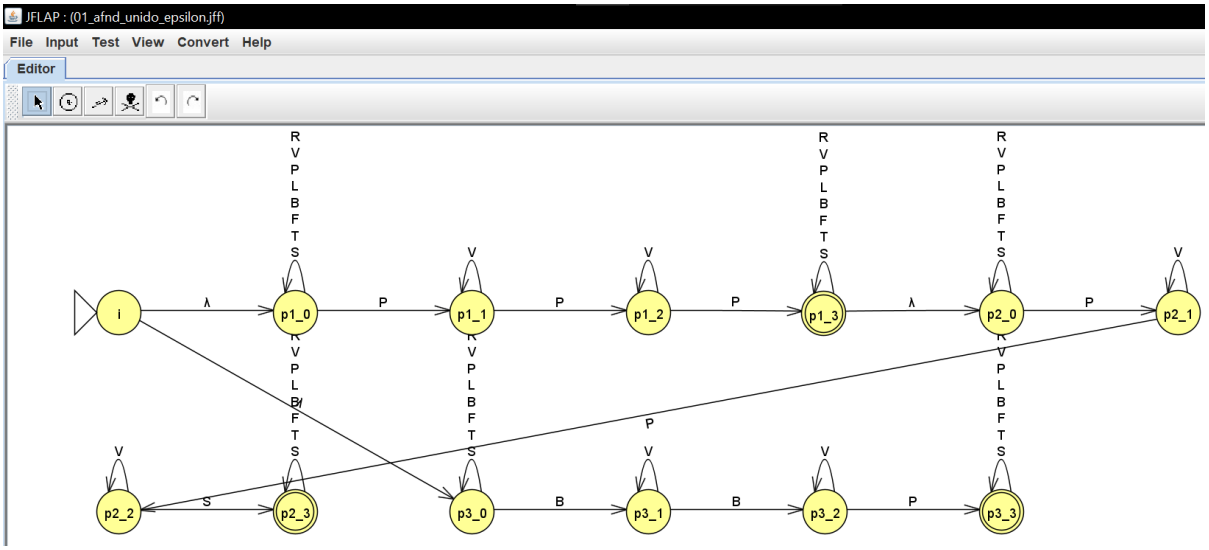


Figura 4. AFND unido con transiciones ε en JFLAP.

Para eliminar las ε: (1) se calculan las clausuras: C(i) = {i, p1_0, p2_0, p3_0} y C(q) = {q} para los demás estados; (2) se calculan las nuevas transiciones δ′(q, a) = C(⋃ r∈C(q) δ(r, a)); (3) un estado es final si su clausura contiene un final, así que F′ = F. Solo cambian las transiciones de i:

Origen	Símbolo	Destinos después de eliminar ε
i	P	{p1_0, p1_1, p2_0, p2_1, p3_0}
i	B	{p1_0, p2_0, p3_0, p3_1}
i	S, T, F, L, V, R	{p1_0, p2_0, p3_0}
cualquier q ≠ i	cualquiera	Igual que en su AFND

Tabla 7. Transiciones de i tras eliminar ε.

El AFND sin ε queda con 13 estados y 90 transiciones: se pierden las 3 ε y el estado i gana 27 transiciones normales.

6. Conversión a AFD y minimización

6.1 Construcción de subconjuntos

Partimos del AFND sin ε con inicial d0 = {i} y D(A, a) = ⋃ q∈A δ′(q, a), generando solo los subconjuntos accesibles con un recorrido en anchura. Un subconjunto es final si contiene algún p_k3. Salieron **41 estados y 328 transiciones**; el AFD es completo y no aparece el conjunto vacío, porque los estados p_k0 tienen bucle con todo Σ. La tabla completa es (→ inicial, * final):

Estado	Subconjunto	S	T	F	B	L	P	V	R
→d0	{i}	d1	d1	d1	d2	d1	d3	d1	d1
d1	{p1_0, p2_0, p3_0}	d1	d1	d1	d2	d1	d3	d1	d1
d2	{p1_0, p2_0, p3_0, p3_1}	d1	d1	d1	d4	d1	d3	d2	d1
d3	{p1_0, p1_1, p2_0, p2_1, p3_0}	d1	d1	d1	d2	d1	d5	d3	d1
d4	{p1_0, p2_0, p3_0, p3_1, p3_2}	d1	d1	d1	d4	d1	d6	d4	d1
d5	{p1_0, p1_1, p1_2, p2_0, p2_1, p2_2, p3_0}	d7	d1	d1	d2	d1	d8	d5	d1
*d6	{p1_0, p1_1, p2_0, p2_1, p3_0, p3_3}	d9	d9	d9	d10	d9	d11	d6	d9
*d7	{p1_0, p2_0, p2_3, p3_0}	d7	d7	d7	d12	d7	d13	d7	d7
*d8	{p1_0, p1_1, p1_2, p1_3, p2_0, p2_1, p2_2, p3_0}	d14	d15	d15	d16	d15	d8	d8	d15
*d9	{p1_0, p2_0, p3_0, p3_3}	d9	d9	d9	d10	d9	d6	d9	d9
*d10	{p1_0, p2_0, p3_0, p3_1, p3_3}	d9	d9	d9	d17	d9	d6	d10	d9
*d11	{p1_0, p1_1, p1_2, p2_0, p2_1, p2_2, p3_0, p3_3}	d18	d9	d9	d10	d9	d19	d11	d9
*d12	{p1_0, p2_0, p2_3, p3_0, p3_1}	d7	d7	d7	d20	d7	d13	d12	d7
*d13	{p1_0, p1_1, p2_0, p2_1, p2_3, p3_0}	d7	d7	d7	d12	d7	d21	d13	d7
*d14	{p1_0, p1_3, p2_0, p2_3, p3_0}	d14	d14	d14	d22	d14	d23	d14	d14
*d15	{p1_0, p1_3, p2_0, p3_0}	d15	d15	d15	d16	d15	d24	d15	d15
*d16	{p1_0, p1_3, p2_0, p3_0, p3_1}	d15	d15	d15	d25	d15	d24	d16	d15
*d17	{p1_0, p2_0, p3_0, p3_1, p3_2, p3_3}	d9	d9	d9	d17	d9	d6	d17	d9
*d18	{p1_0, p2_0, p2_3, p3_0, p3_3}	d18	d18	d18	d26	d18	d27	d18	d18
*d19	{p1_0, p1_1, p1_2, p1_3, p2_0, p2_1, p2_2, p3_0, p3_3}	d28	d29	d29	d30	d29	d19	d19	d29
*d20	{p1_0, p2_0, p2_3, p3_0, p3_1, p3_2}	d7	d7	d7	d20	d7	d27	d20	d7
*d21	{p1_0, p1_1, p1_2, p2_0, p2_1, p2_2, p2_3, p3_0}	d7	d7	d7	d12	d7	d31	d21	d7
*d22	{p1_0, p1_3, p2_0, p2_3, p3_0, p3_1}	d14	d14	d14	d32	d14	d23	d22	d14
*d23	{p1_0, p1_1, p1_3, p2_0, p2_1, p2_3, p3_0}	d14	d14	d14	d22	d14	d31	d23	d14
*d24	{p1_0, p1_1, p1_3, p2_0, p2_1, p3_0}	d15	d15	d15	d16	d15	d8	d24	d15
*d25	{p1_0, p1_3, p2_0, p3_0, p3_1, p3_2}	d15	d15	d15	d25	d15	d33	d25	d15
*d26	{p1_0, p2_0, p2_3, p3_0, p3_1, p3_3}	d18	d18	d18	d34	d18	d27	d26	d18
*d27	{p1_0, p1_1, p2_0, p2_1, p2_3, p3_0, p3_3}	d18	d18	d18	d26	d18	d35	d27	d18
*d28	{p1_0, p1_3, p2_0, p2_3, p3_0, p3_3}	d28	d28	d28	d36	d28	d37	d28	d28
*d29	{p1_0, p1_3, p2_0, p3_0, p3_3}	d29	d29	d29	d30	d29	d33	d29	d29
*d30	{p1_0, p1_3, p2_0, p3_0, p3_1, p3_3}	d29	d29	d29	d38	d29	d33	d30	d29
*d31	{p1_0, p1_1, p1_2, p1_3, p2_0, p2_1, p2_2, p2_3, p3_0}	d14	d14	d14	d22	d14	d31	d31	d14
*d32	{p1_0, p1_3, p2_0, p2_3, p3_0, p3_1, p3_2}	d14	d14	d14	d32	d14	d37	d32	d14
*d33	{p1_0, p1_1, p1_3, p2_0, p2_1, p3_0, p3_3}	d29	d29	d29	d30	d29	d19	d33	d29
*d34	{p1_0, p2_0, p2_3, p3_0, p3_1, p3_2, p3_3}	d18	d18	d18	d34	d18	d27	d34	d18
*d35	{p1_0, p1_1, p1_2, p2_0, p2_1, p2_2, p2_3, p3_0, p3_3}	d18	d18	d18	d26	d18	d39	d35	d18
*d36	{p1_0, p1_3, p2_0, p2_3, p3_0, p3_1, p3_3}	d28	d28	d28	d40	d28	d37	d36	d28
*d37	{p1_0, p1_1, p1_3, p2_0, p2_1, p2_3, p3_0, p3_3}	d28	d28	d28	d36	d28	d39	d37	d28
*d38	{p1_0, p1_3, p2_0, p3_0, p3_1, p3_2, p3_3}	d29	d29	d29	d38	d29	d33	d38	d29
*d39	{p1_0, p1_1, p1_2, p1_3, p2_0, p2_1, p2_2, p2_3, p3_0, p3_3}	d28	d28	d28	d36	d28	d39	d39	d28
*d40	{p1_0, p1_3, p2_0, p2_3, p3_0, p3_1, p3_2, p3_3}	d28	d28	d28	d40	d28	d37	d40	d28

Tabla 8. Construcción de subconjuntos (solo estados accesibles).

Por ejemplo, d0 con B va a d2, d0 con P va a d3, d2 con B va a d4, d3 con P va a d5, y d4 con P o d5 con S o P llegan a un estado final. Los 35 estados finales (los que contienen p1_3, p2_3 o p3_3) aparecen porque, una vez detectado un patrón, el AFD sigue llevando la cuenta de los otros dos.

6.2 Minimización

Usamos refinamiento de particiones: se empieza con finales y no finales y cada bloque se divide según a qué bloques van sus estados con cada símbolo. El número de bloques pasó de 2 a 4 y luego a 6, y ahí se estabilizó. Las clases no finales son m1 = {d0, d1}, m2 = {d2}, m3 = {d3}, m4 = {d4} y m5 = {d5}; los 35 finales se juntan en m0. El AFD mínimo es **M_min = ({m0, …, m5}, Σ, δ, m1, {m0})**, con 6 estados y 48 transiciones:

Estado	Qué recuerda	S	T	F	B	L	P	V	R
*m0	Patrón detectado	m0	m0	m0	m0	m0	m0	m0	m0
→m1	Nada útil todavía	m1	m1	m1	m2	m1	m3	m1	m1
m2	Último símbolo B	m1	m1	m1	m4	m1	m3	m2	m1
m3	Último símbolo P	m1	m1	m1	m2	m1	m5	m3	m1
m4	Últimos dos BB	m1	m1	m1	m4	m1	m0	m4	m1
m5	Últimos dos PP	m0	m1	m1	m2	m1	m0	m5	m1

Tabla 9. AFD mínimo y lo que recuerda cada estado.

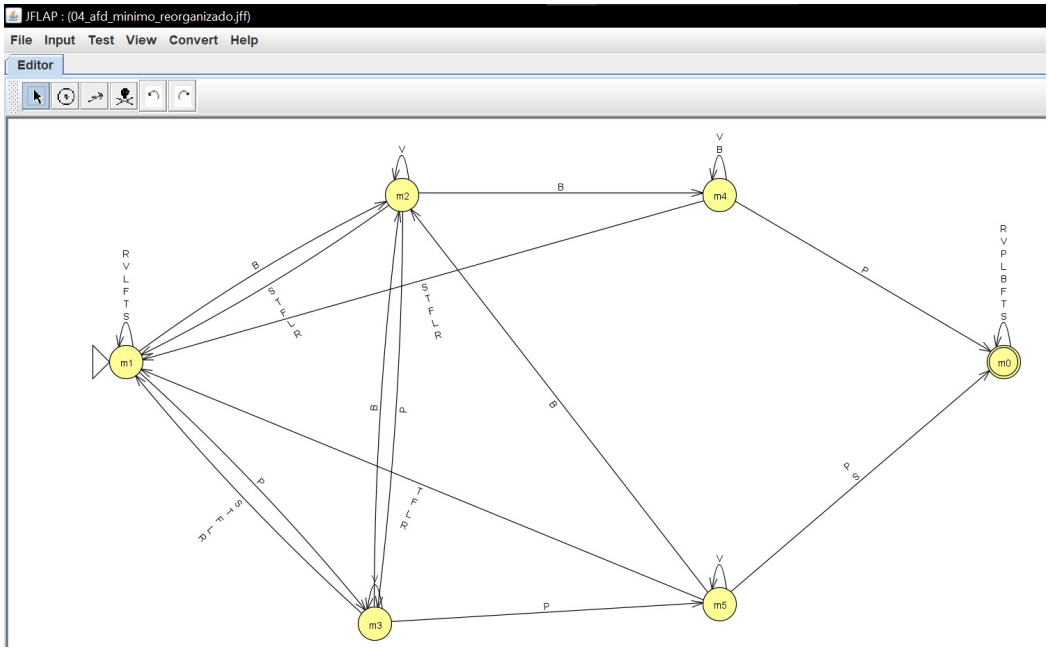


Figura 5. AFD mínimo en JFLAP.

Es mínimo porque cada par de estados se distingue con alguna cadena: m0 con la cadena vacía (es el único final), m1 y m2 con BP, m1 y m3 con PS, y m4 y m5 con S. Todos son alcanzables: m2 con B, m3 con P, m4 con BB, m5 con PP y m0 con PPP. La equivalencia entre el AFD de 41 estados y el mínimo se comprobó en Python con el autómata producto. La misma comprobación puede repetirse en JFLAP con Test → Compare Equivalence entre 03_afd.jff y 04_afd_minimo.jff.

7. Implementación del monitor

El monitor está en Python (pandas 2.0 o superior). monitor.py carga el AFD mínimo desde un JSON y no usa las búsquedas de texto con las que se etiquetaron los datos, así la evaluación es independiente.

Archivo	Qué hace
datos/preparar_datos.py	Limpieza, casos cliente-año, codificación, muestreo y claves

Archivo	Qué hace
monitor/automata.py	Clausura ε, simulación del AFND, eliminación de ε, subconjuntos, minimización, simulación del AFD y equivalencia
monitor/monitor.py	Lee el CSV, ejecuta el AFD mínimo y calcula las métricas
monitor/verificar.py	Pruebas reproducibles de los modelos
automatas/*.json y *.jff	Autómatas para Python y para JFLAP

Tabla 10. Módulos del proyecto.

Para cada caso: se comprueba que estén las columnas y que no haya id_evento repetidos, se ordenan los eventos por fecha, se forma la cadena w, se empieza en m1, se aplica una transición por símbolo y se clasifica según si termina en m0. El estado se reinicia en cada caso. Si aparece un símbolo fuera de Σ, el caso se marca como no evaluable en vez de contarse como alerta. El AFD procesa cada cadena en tiempo O(n) con memoria constante. Se ejecuta con: python monitor/monitor.py datos/validacion.csv --clave datos/clave_validacion.csv --salida detecciones.csv.

Como el AFD mínimo tiene un solo estado final, solo dice si hay alerta. Para saber qué patrón la disparó se pueden consultar los tres AFND por separado.

8. Resultados

Conjunto	VP	FP	FN	VN	Precisión	Exhaustividad
Validación (300)	54	0	0	246	1,000	1,000
Prueba (50)	9	0	0	41	1,000	1,000

Tabla 11. Métricas del monitor.

Autómata	Estados	Transiciones
AFND P1 / P2 / P3	4 cada uno	21 cada uno
Unión con ε	13	66
Sin ε	13	90
AFD (subconjuntos)	41	328
AFD mínimo	6	48

Tabla 12. Tamaño de cada autómata.

Tiempos. Con las 350 cadenas (siete repeticiones de diez pasadas, sin contar la lectura del archivo), la mediana fue de 2,033 ms para el AFND con ε y de 0,149 ms para el AFD mínimo, unas 14 veces menos. El AFND mantiene un conjunto de estados activos y calcula clausuras en cada símbolo; el AFD hace una sola consulta. Los tiempos dependen de la máquina.

Pruebas adicionales. Comparamos el AFD con el oráculo de texto y con el AFND en las 4 681 cadenas posibles de longitud 0 a 4 y en 1 000 cadenas aleatorias de longitud 5 a 100, sin ninguna diferencia. Las cadenas PVPP, PPVS y BVBVP confirman que V no rompe las rachas.

Cómo leer el 100 %. Las etiquetas son reglas calculadas sobre π(w), no casos de fraude confirmados por una persona. El resultado demuestra que el autómata implementa exactamente las reglas, no que cada alerta sea un fraude real.

9. Límites del modelo

Una regla que depende del tiempo. «Los tres fallos ocurrieron en menos de cinco minutos» no la puede verificar el AFD: dos clientes pueden tener la misma cadena PPP con tiempos muy distintos entre fallos, y el autómata recibe exactamente la misma entrada. Se podría adaptar con un preprocesador que agregue un símbolo cuando pasa cierto tiempo, como hace la guía con el

evento de tiempo excedido; no lo implementamos para no agregar eventos que no están en la fuente.

Una regla que necesita contar sin límite. «Un cliente no puede tener más pagos fallidos que compras exitosas» corresponde al lenguaje $L = \{w \in \Sigma^* \mid \#P(w) \leq \#S(w)\}$, que no es regular. Si lo fuera, tendría una constante p del lema de bombeo; tomamos $w = S^pP^p$, que está en L. Toda división w = xyz con $|xy| \leq p$ y $|y| \geq 1$ deja y formada solo por S, y al bombear hacia abajo (xz) quedan menos S que P, así que $xz \notin L$: contradicción [4]. El autómata tendría que llevar la cuenta de la diferencia entre fallos y éxitos, y esa diferencia puede crecer sin tope.

Límites de los datos. La fuente no trae los pasos de la compra ni un identificador de sesión, así que un caso cliente-año no es una compra ni una sesión interactiva. Tampoco pudimos verificar quién publicó originalmente los datos ni su licencia, y otro repositorio que usa las mismas cinco tablas indica que fueron simuladas para un proyecto de aprendizaje [7]. Por eso no afirmamos que sean datos reales de una empresa.

Límites de la evaluación. Los patrones se eligieron explorando los mismos datos, y la muestra tiene 18 % de alertas por diseño frente al 3,89 % del total. P3 tiene solo tres casos. Falta que una persona revise las etiquetas y que la docente confirme si acepta la agrupación cliente-año.

10. Conclusiones

A la pregunta guía: en los resultados de pago conviene vigilar las rachas de fallos, las compras que salen bien justo después de varios fallos y las faltas de respuesta del banco seguidas de fallo. Las tres se escriben como lenguajes regulares y un AFD mínimo de 6 estados las detecta en una sola lectura de cada caso.

- Diseñar con AFND fue más simple: cada patrón cabe en 4 estados. La construcción de subconjuntos llevó la unión a 41 estados y la minimización la dejó en 6, que se pueden explicar uno por uno (tabla 9).
- El AFD fue unas 14 veces más rápido que el AFND en nuestras mediciones.
- El evento irrelevante importa: sin el bucle de V, una verificación de cuenta en medio de dos fallos ocultaría una racha.
- El monitor detecta patrones de seguimiento, no fraude confirmado, y depende de la calidad y procedencia de los datos.

Como mejoras proponemos conseguir bitácoras con procedencia y licencia verificadas, que incluyan los pasos de la compra; validar las reglas con alguien del negocio, y evaluar el monitor con datos de un periodo posterior que no se haya usado para elegir los patrones.

Referencias

[1] Universidad Nacional de Loja, «Evaluación sumativa 1 · Unidad 1: Análisis de secuencias de eventos con autómatas finitos. Lineamientos y plantillas», Teoría de Autómatas y Computabilidad Avanzada, periodo septiembre 2026 – marzo 2027.

[2] N. Thuy Linh, «Movie Ticket Booking Analysis», repositorio Movie_Ticket_Python en GitHub. [En línea]. Disponible: https://github.com/ngthuylinh3003/Movie_Ticket_Python

[3] «ticket_history.csv» y «status_detail.csv», archivos de datos enlazados desde [2]. Descargados el 28 de octubre de 2026.

[4] J. Esparza y M. Blondin, *Automata Theory: An Algorithmic Approach*. Cambridge, MA, USA: MIT Press, 2023.

[5] OWASP Foundation, «OAT-001 Carding», *OWASP Automated Threats to Web Applications*. [En línea]. Disponible: https://wiki.owasp.org/index.php/OAT-001_Carding

[6] Stripe, «Protect yourself from card testing», *Stripe Documentation*. [En línea]. Disponible: https://docs.stripe.com/card-testing

[7] QL-KHANG, «movie-ticket-analysis», repositorio en GitHub. [En línea]. Disponible: https://github.com/QL-KHANG/movie-ticket-analysis

Fuentes en línea consultadas el 28 de octubre de 2026. Las citas en inglés están traducidas.

Declaración de uso de IA

Usamos Perplexity para interpretar los requisitos, organizar la búsqueda de datos, proponer la codificación y las reglas, escribir el código de las construcciones y los simuladores, ejecutar las pruebas y generar los archivos de JFLAP. Usamos Claude (Anthropic) para revisar el informe, verificar los números recalculándolos desde los CSV, corregir inconsistencias y buscar fuentes que respaldaran los patrones. Las métricas salen de ejecutar el código, y cualquier integrante puede trazar a mano una cadena en los autómatas.